SIMATS ENGINEERING COLLEGE

C PROGRAMMING LAB — CO1 ASSIGNMENT 3

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Complete worked report: design, C source code, test cases, numerical analysis, recommendations and smart-meter redesign

Student Name	_____
Register Number	_____
Course / Section	_____
GitHub Repository Link	_____

Submission date shown in the assignment: 27-10-2026

1. Problem Understanding and Design

Grama Jyothi Rural Electricity Co-operative bills households using a fixed charge and a slab tariff. The program must avoid incorrect bills caused by floating-point rounding, meter rollover, invalid readings, and totals that exceed the range of small integer types.

Tariff rules used in this report

Item	Rule
Fixed charge	Rs. 40.00 per bill
First slab	First 100 units are free
Second slab	Units 101–200 cost Rs. 2.25 per unit
Third slab	Units above 200 cost Rs. 4.50 per unit
Meter rollover	If the meter changes from 65535 to 0, treat it as one rollover; units = (65535 – previous) + 1 + current
Money representation	Use integer paise for all bill calculations; convert to rupees only for display

Assumption: the tariff examples in the prompt are interpreted as progressive slabs, not as one rate applied to all consumed units. A bill is fixed charge + slab charges. For the smart-meter redesign, the 20% peak-time premium is applied to the energy charge for the units recorded between 6 PM and 10 PM; the fixed charge is not increased.

Chosen data types and constants

Item	Choice	Reason
Meter readings	unsigned long	Can store non-negative readings and scale to many readings
Units and counters	unsigned long long	Provides wider range for accumulated energy and district totals
Money	long long integer paise	Avoids binary floating-point rounding in billing
Slab prices	225 paise and 450 paise	Exact representation of Rs. 2.25 and Rs. 4.50
Maximum meter reading	65535UL	Matches the rollover boundary in the assignment

2. Complete C Program (Task 1)

The following program validates inputs, handles a single meter rollover, computes progressive slabs in paise, and prints the bill. It accepts previous and current meter readings as command-line input values typed when prompted.

```
#include <stdio.h>
#include <limits.h>

#define FIXED_CHARGE_PAISE 4000LL
#define FREE_UNITS 100ULL
#define SECOND_SLAB_LIMIT 200ULL
#define SECOND_RATE_PAISE 225LL
#define THIRD_RATE_PAISE 450LL
#define METER_MAX 65535UL

/*
 * Return 1 if readings are valid and units are calculated.
 * A decrease is accepted only when it is a single 65535 -> 0 rollover.
 */
int calculate_units(unsigned long previous, unsigned long current,
                    unsigned long long *units)
{
    if (previous > METER_MAX || current > METER_MAX)
        return 0;

    if (current >= previous) {
        *units = (unsigned long long)(current - previous);
        return 1;
    }

    /* Valid single rollover of a 16-bit meter. */
    *units = (unsigned long long)(METER_MAX - previous)
             + 1ULL + (unsigned long long)current;
```

```
    return 1;
}

/* Return the total bill in paise using progressive slabs. */
long long calculate_bill_paise(unsigned long long units)
{
    unsigned long long second_units = 0ULL;
    unsigned long long third_units = 0ULL;
    long long energy_charge = 0LL;

    if (units > FREE_UNITS) {
        if (units <= SECOND_SLAB_LIMIT) {
            second_units = units - FREE_UNITS;
        } else {
            second_units = SECOND_SLAB_LIMIT - FREE_UNITS;
            third_units = units - SECOND_SLAB_LIMIT;
        }
    }

    energy_charge += (long long)second_units * SECOND_RATE_PAISE;
    energy_charge += (long long)third_units * THIRD_RATE_PAISE;

    return FIXED_CHARGE_PAISE + energy_charge;
}

int main(void)
{
    unsigned long previous, current;
    unsigned long long units;
    long long bill_paise;

    printf("Grama Jyothi Electricity Billing\\n");
    printf("Enter previous meter reading (0-65535): ");

    if (scanf("%lu", &previous) != 1) {
        printf("Error: reading must be a number.\\n");
        return 1;
    }

    printf("Enter current meter reading (0-65535): ");
    if (scanf("%lu", &current) != 1) {
        printf("Error: reading must be a number.\\n");
        return 1;
    }

    if (previous > METER_MAX || current > METER_MAX) {
        printf("Error: meter reading must be between 0 and 65535.\\n");
        return 1;
    }

    if (current < previous && !(previous == METER_MAX && current == 0UL)) {
        printf("Error: current reading is lower than previous reading; "
               "no valid rollover was detected.\\n");
        return 1;
    }

    if (!calculate_units(previous, current, &units)) {
        printf("Error: invalid meter readings.\\n");
        return 1;
    }

    bill_paise = calculate_bill_paise(units);

    printf("\\nUnits consumed: %llu\\n", units);
    printf("Fixed charge: Rs. 40.00\\n");
    printf("Energy charge: Rs. %lld.%02lld\\n",
           (bill_paise - FIXED_CHARGE_PAISE) / 100LL,
           (bill_paise - FIXED_CHARGE_PAISE) % 100LL);
    printf("Total bill: Rs. %lld.%02lld\\n",
           bill_paise / 100LL, bill_paise % 100LL);

    return 0;
}
```

Compile and run (GCC): `gcc billing.c -std=c11 -Wall -Wextra -Wconversion -o billing`, then run `./billing` on Linux/macOS or `billing.exe` on Windows.

Input validation note: this version allows a lower current reading only for the exact 65535-to-0 rollover specified in the assignment. If the real meter can roll over and show a non-zero current reading, the condition can be broadened to accept any decrease as a rollover, but that can also hide a data-entry mistake. A production system should store a rollover counter or meter event flag.

3. Hand-calculated Test Cases and Expected Results

The table below provides a small set of tests. Expected totals include the Rs. 40 fixed charge.

Test	Previous	Current	Units	Energy charge	Total bill	Expected behaviour
Zero use	1200	1200	0	Rs. 0.00	Rs. 40.00	Valid
Exactly 100	1000	1100	100	Rs. 0.00	Rs. 40.00	Valid boundary
Exactly 200	1000	1200	200	Rs. 225.00	Rs. 265.00	100 × Rs. 2.25
250 units	1000	1250	250	Rs. 450.00	Rs. 490.00	100 × 2.25 + 50 × 4.50
Rollover	65535	0	1	Rs. 0.00	Rs. 40.00	Valid rollover
Invalid decrease	500	400	—	—	—	Rejected
Out of range	70000	70010	—	—	—	Rejected
Non-numeric	abc	120	—	—	—	Rejected

Example calculation for 250 units: the first 100 units are free. The next 100 units cost 100 × Rs. 2.25 = Rs. 225.00. The remaining 50 units cost 50 × Rs. 4.50 = Rs. 225.00. Energy charge = Rs. 450.00. Add Rs. 40.00 fixed charge: total = Rs. 490.00.

For the exact rollover case, consumed units = (65535 – 65535) + 1 + 0 = 1 unit. Because one unit is inside the free slab, the bill is Rs. 40.00.

4. Task 1 — Alternative Concept and Complexity

Alternative design: store slab limits and rates in arrays, then loop through the slabs instead of writing separate if-statements. For s slabs, the calculation takes O(s) time and O(s) extra space for the arrays. With a fixed three-slab tariff, the current if-statement design takes O(1) time and O(1) extra space.

Comparison	Fixed if-statements	Arrays + loop
Runtime per bill	O(1)	O(s)
Extra tariff storage	O(1)	O(s)
Ease of changing tariff	Need code edits	Update array values / configuration
Best use	Very small, fixed tariff	Many slabs or frequently revised tariffs

Recommendation: use arrays or a tariff configuration file when the tariff is likely to change. For only three fixed slabs, both methods are fast; maintainability is the main difference.

5. Task 2 — Expected Versus Observed Execution and Numeric Types

The hand calculations and the program should agree for every valid test. The design uses integer paise, so Rs. 2.25 is stored as 225 paise and Rs. 4.50 as 450 paise. This prevents results such as Rs. 224.999999 caused by binary floating-point representation.

For 4,000, 60,000 and 800,000 households, the largest example bill in the tests is Rs. 490.00. If every household had that bill, totals would be:

Households	Total in rupees	Total in paise
4,000	1,960,000.00	196,000,000
60,000	29,400,000.00	2,940,000,000
800,000	392,000,000.00	39,200,000,000

These totals are illustrative worst-case totals based on every household having a Rs. 490 bill; actual totals depend on actual consumption. A 32-bit signed int cannot safely hold 39.2 billion paise. A 64-bit long long is the safer choice for accumulated money.

C type discussion: the C standard guarantees minimum ranges, not that int is always 32 bits or long is always 64 bits. On many 64-bit Windows compilers, long is still 32 bits; on common 64-bit Linux systems, long is 64 bits. long long is at least 64 bits. Use limits from `limits.h` and check multiplication/addition bounds if a production system accepts unbounded input.

Float-versus-integer comparison: a float or double can represent many decimal values only approximately. Summing a large number of bill amounts can accumulate small rounding differences. Integer paise is exact for two-decimal currency, provided overflow is prevented. A double accumulator is easy to use but requires rounding rules and is not preferred for financial totals.

Alternative Concept and Complexity Analysis

If the district total were accumulated in double instead of long long, it would not normally overflow at this scale, but it could introduce rounding error because decimal currency values are not always represented exactly in binary. Each addition is O(1), and the total operation over n bills is O(n) for either type. Storage per accumulator is O(1). The primary reason to choose long long paise is exactness, not speed.

6. Task 3 — Final Design Recommendation

Recommended design for the current assignment:

Design choice	Recommendation
Meter readings	unsigned long, validated against the 0–65535 meter range
Consumed units	unsigned long long
Currency	long long paise; format as rupees and paise only when printing
Constants	Use named #define constants for fixed charge, slab limits and rates
Tariff revision	Move slab limits/rates into a configuration file or validated tariff table
Input safety	Reject letters, negative readings, out-of-range readings and unexplained decreases
Testing	Keep repeatable boundary, rollover, invalid-input and manually calculated tests

Limitations: the sample program processes one household at a time, assumes the specified 16-bit meter range and exact rollover condition, and does not store customer records or write bills to a database. District-scale use needs persistent storage, duplicate-reading checks, audit logs, access controls, backup, and explicit handling for meter replacements and multiple rollovers.

Tariff revision approach: store validated rates and slab limits in a configuration file or database table, load them when the program starts, and record a version/effective date for each tariff. Validate that limits increase and rates are non-negative. This avoids recompilation for every tariff change, but it adds file I/O, parsing, validation and a risk of a missing or malformed configuration file.

Evidence expected by the assignment: include your own terminal screenshots showing compiler version, compile command, test input, actual output and a comparison with the expected table. Do not claim measured execution results unless you have actually run the program.

7. Task 4 — Smart-Meter Design Challenge

Changed requirement: smart meters send energy readings in watt-hours every 15 minutes, producing 35,040 readings per household per year. Between 6 PM and 10 PM, units cost 20% more. Tariff values must be changeable without recompiling.

Recommended redesign

Retain the core billing rules but modify the data model and tariff handling. Store timestamped watt-hour readings, calculate interval consumption as current cumulative watt-hours minus the previous value, and convert to kWh by dividing watt-hours by 1,000. For billing, sum the energy used in each time period and apply the 20% premium only to intervals whose timestamps fall in the 6 PM–10 PM window. The system must define the local time zone and how daylight-saving changes are handled where applicable.

Because a year has 365 × 24 × 4 = 35,040 quarter-hour intervals, 4,000 households produce 140,160,000 interval readings per year; 60,000 households produce 2,102,400,000; and 800,000 households produce 28,032,000,000. This scale needs streaming or batch processing, indexed storage, and aggregation rather than keeping all readings in a small in-memory array.

Households	Readings per year per household	Total readings per year
4,000	35,040	140,160,000
60,000	35,040	2,102,400,000
800,000	35,040	28,032,000,000

At a nominal 8 bytes per reading value alone, 800,000 households × 35,040 readings × 8 bytes is about 224.3 GB per year, before timestamps, customer IDs, indexes, replication and backups. Real storage will be higher. Storing only interval consumption, compact timestamps, and aggregated billing-period summaries can reduce cost; raw readings may still need retention for audits.

Peak-time premium calculation

Example only: if a household uses 10 kWh off-peak and 5 kWh during the peak window, and the base rate for those units is Rs. 4.50/kWh, off-peak energy charge = 10 × 4.50 = Rs. 45.00. Peak energy charge = 5 × 4.50 × 1.20 = Rs. 27.00. Combined energy charge = Rs. 72.00, plus the fixed charge and any applicable slab calculation. For the real tariff, the co-operative must specify whether the peak premium is applied before or after progressive slab allocation. This report assumes the premium is applied to the energy charge for peak-period units after identifying the relevant base slab rate.

Tariff configuration should contain the effective date, slab limits, base rates, fixed charge, peak premium and peak-time window. The program should load the active configuration at startup, validate it, and log the version used for each bill. If loading fails, it should stop safely rather than silently use a partial tariff.

Retain, modify or redesign?

Decision: MODIFY the original design. Retain the reliable core ideas—integer paise, input validation, named constants, boundary testing and wide accumulators. Modify the single-reading input into timestamped interval processing, move rates to validated configuration, and add peak/off-peak classification. Redesign data storage and processing for district scale because tens of billions of annual readings cannot be handled efficiently as one in-memory list.

Trade-off: reading a configuration file adds a small startup cost and error-handling code, but avoids recompiling and redistributing the program for each tariff revision. For a district system, this is preferable to hard-coded rates. The billing engine should use a versioned configuration so a bill can later be reproduced using the exact tariff that applied.

8. Conclusion

The program meets the main single-household requirements by computing progressive slabs, adding the fixed charge, rejecting invalid input and handling the specified meter rollover. Integer paise avoids ordinary currency rounding errors, while long long supports large accumulated totals more safely than int. For smart meters, the best direction is a modified billing engine with interval readings, peak-time pricing, validated external tariff configuration and scalable storage. Before submission, compile and run the code on your own lab computer, attach actual output screenshots, and add the GitHub repository link requested by the lab.

9. Submission Checklist

• Create a GitHub repository named CSA02--Assignment1 (use the exact naming rule from the lab portal).

- Keep source code, input files, output screenshots and notes in a task folder.
- Add README.md with name, register number, compiler/version, compile command and short test notes.
- Commit progress in multiple commits; do not upload everything only on the last day.
- Upload the PDF report and verify that the GitHub link is working before saving the lab submission.
- Replace the blank name/register fields on the cover and insert your actual repository link.